EXPERIMENT-01

Exploring Matplotlib, Seaborn, and Plotly

AIM:

To perform data exploration, cleaning, and visualization on a large real-world dataset (Diamond dataset) using Matplotlib, Seaborn, and Plotly, and to derive analytical insights from different graph types.

LIBRARIES:

- Pandas
- Matplotlib
- Seaborn
- Plotly Express

PROCEDURE:

1. Read the dataset from the URL using pandas.read_csv().

2. Check missing values with df.isnull().sum() and drop null rows.

3. Remove irrelevant columns like 'depth' and 'table'.

4. Take a sub-sample of 3,000 rows for fast Plotly interactive plots.

5. Generate Bar Plot, Scatter Plot, Pie Chart, Histogram, and Box Plot using Matplotlib, Seaborn, and Plotly.

6. Derive insights from each generated chart.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

# 1. Dataset Loading
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/diamonds.csv"
df = pd.read_csv(url)

# 2. Data Exploration & Inspection
print("--- DATA EXPLORATION ---")
print("Column Names:\n", df.columns.tolist())
print("Dataset Shape:", df.shape)
print("\nMissing Values Per Column:\n", df.isnull().sum())

# 3. Data Cleaning & Sub-sampling
```

```
df_cleaned = df.dropna().drop(columns=['depth', 'table']) df_sample =
df_cleaned.sample(n=3000, random_state=42)

print("\n--- GENERATING PLOTS ---")

# Plot 1: Bar Plot (Seaborn) - Average Price by Cut plt.figure(figsize=(8, 5))
sns.barplot(data=df_cleaned, x='cut', y='price', palette='viridis', errorbar=None) plt.title('Average Diamond Price by Cut Quality')
plt.xlabel('Cut') plt.ylabel('Average Price ($)')
plt.grid(axis='y', linestyle='--', alpha=0.7) plt.tight_layout()
plt.show()

# Plot 2: Interactive Scatter Plot (Plotly) - Carat vs Price fig_scatter = px.scatter(
        df_sample, x='carat',
        y='price', color='clarity',
        title='Diamond Carat vs Price (Sampled Interactive Scatter)', labels={'carat': 'Carat
        Weight', 'price': 'Price ($)'}, opacity=0.7
)
fig_scatter.show()

# Plot 3: Pie Chart (Matplotlib) - Proportion of Cuts cut_counts =
df_cleaned['cut'].value_counts() plt.figure(figsize=(7, 7))
plt.pie(
        cut_counts, labels=cut_counts.index,
        autopct='%1.1f%%', startangle=140,
        colors=sns.color_palette('pastel')
)
plt.title('Proportion of Diamond Cuts Available') plt.tight_layout()
plt.show()

# Plot 4: Histogram (Seaborn) - Price Distribution plt.figure(figsize=(8, 5))
sns.histplot(data=df_cleaned, x='price', kde=True, bins=50, color='Crimson') plt.title('Distribution of Diamond Prices')
plt.xlabel('Price ($)')
plt.ylabel('Frequency')
plt.tight_layout() plt.show()
```

```
# Plot 5: Interactive Box Plot (Plotly) - Price Across Colors
fig_box = px.box(
    df_sample,
    x='color',
    y='price',
    color='color',
    title='Price Distribution Across Diamond Colors (Box Plot)',
    category_orders={'color': ['D', 'E', 'F', 'G', 'H', 'I', 'J']}
)
fig_box.show()
```

VISUALIZATION & ANALYTICAL INSIGHTS:

1. Ideal Cuts vs Fair Cuts: Ideal cuts show a lower average price than 'Fair' cuts because they tend to be smaller in carat weight.

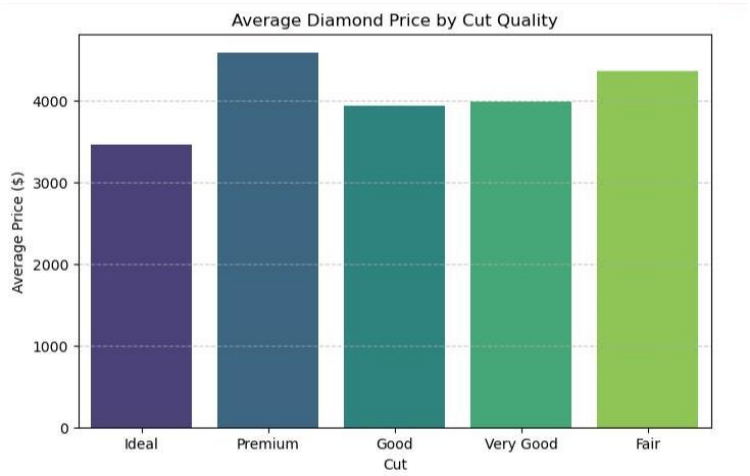
2. Price vs Carat Weight: Price increases exponentially with carat weight, with higher clarity grades sitting at the top price levels.

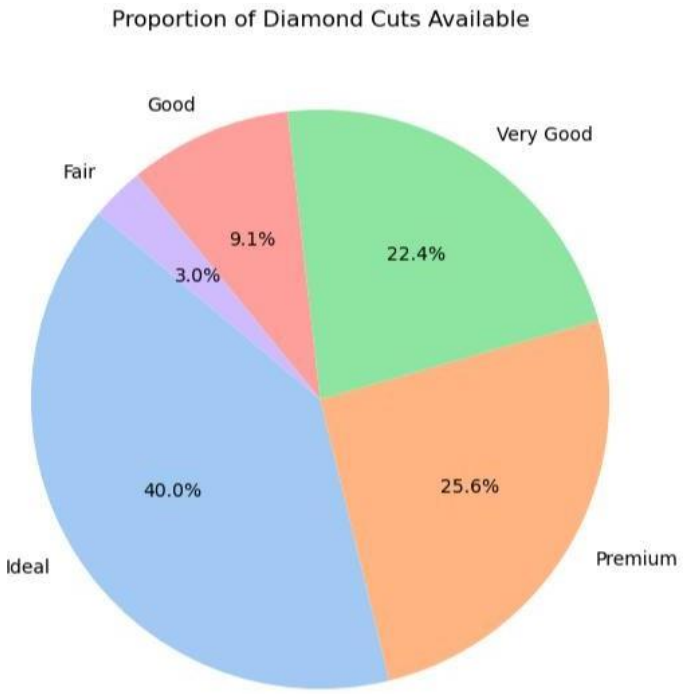
3. Cut Proportions: Over 40% of the inventory consists of 'Ideal' cuts, while 'Fair' cuts make up only about 3%.

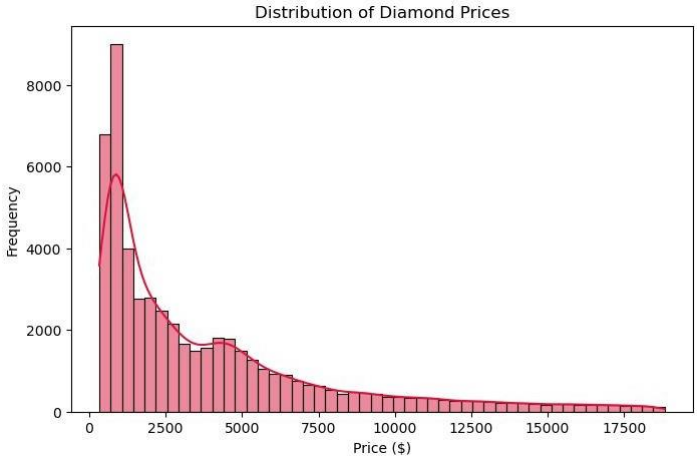
4. Price Distribution: Prices are strongly right-skewed, peaking heavily under $2,000 with a long thin tail extending to $18,000.

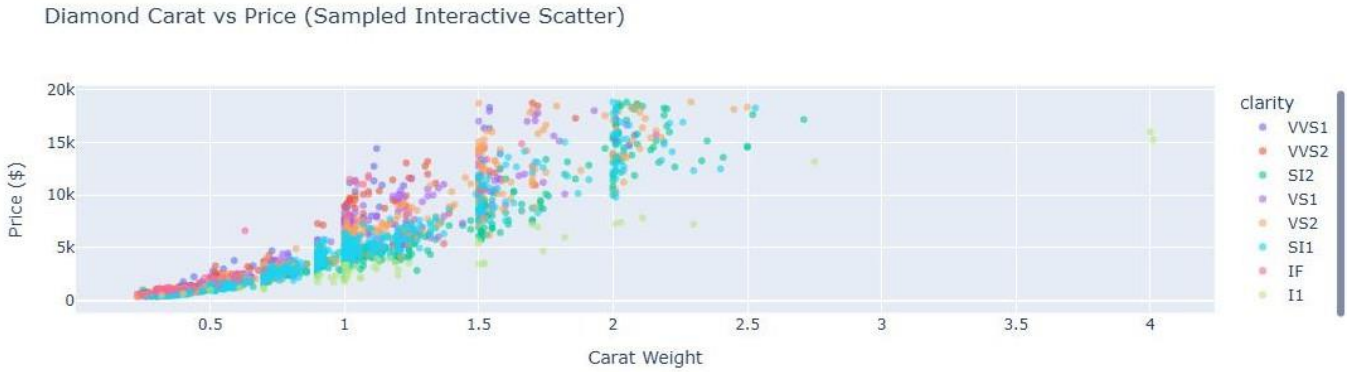
5. Color vs Price Trend: Median price and spread increase as color grade drops (D to J) due to lower color grade diamonds being larger on average.

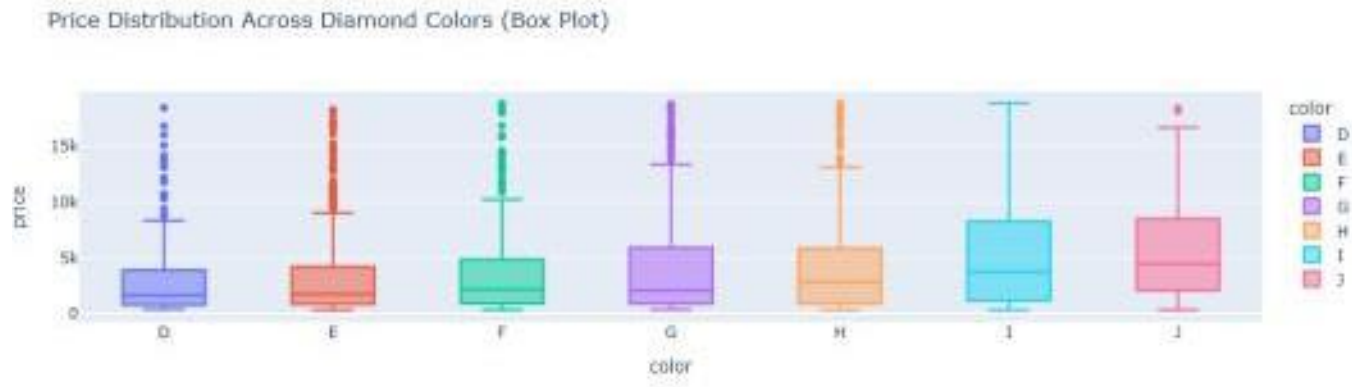
OUTPUT:


Average Diamond Price by Cut Quality
Average Price ($)
0
1000
2000
3000
4000
Ideal
Premium
Good
Very Good
Fair
Cut


Proportion of Diamond Cuts Available
Good
Very Good
Fair
9.1%
22.4%
3.0%
40.0%
25.6%
Ideal
Premium







RESULT:

Data exploration, cleaning, and visualization were successfully completed on the Diamond dataset using Matplotlib, Seaborn, and Plotly, and key analytical insights were derived.

EXPERIMENT-02

PYTHON WIDGETS DASHBOARD USING IPYWIDGETS

AIM

To create an interactive health and fitness dashboard using Python, Pandas, Matplotlib, and Jupyter Widgets for analyzing and visualizing wearable health telemetry data.

PROCEDURE

1. Import the required Python libraries.
2. Generate a synthetic health telemetry dataset using Python and NumPy.
3. Create interactive widgets for activity, step goal, chart type, and sleep threshold.
4. Filter the dataset based on the selected widget values.
5. Calculate summary health metrics.
6. Visualize the data using interactive Matplotlib charts.
7. Display the dashboard and update it dynamically based on user selections.

CODE

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, ipywidgets as widgets
from IPython.display import display, clear_output

np.random.seed(42)
n_days = 60
dates = pd.date_range(end=pd.Timestamp.now(), periods=n_days, freq="D")
activities = ["Running", "Cycling", "Swimming", "HIIT", "Walking"]

df = pd.DataFrame({
    "Date": dates,
    "Activity": np.random.choice(activities, n_days),
    "StepCount": np.random.randint(3000, 16000, n_days),
    "AvgHeartRate": np.random.randint(60, 165, n_days),
    "CaloriesBurned": np.random.randint(200, 900, n_days),
    "SleepScore": np.random.randint(50, 100, n_days)
})

activity = widgets.SelectMultiple(options=activities, value=activities,
```

```
description="Activities: ")
steps = widgets.IntSlider(value=5000, min=3000, max=12000, step=1000, description="Min Steps Goal: ")
chart = widgets.RadioButtons(
    options=["Heart Rate vs Calories (Scatter)", "Daily Step Trends (Line)", "Sleep Quality (Bar)"],
    value="Heart Rate vs Calories (Scatter)", description="Metric View: "
)
sleep_check = widgets.Checkbox(value=True, description="Highlight Optimal Sleep Range (>=75)")

output = widgets.Output() def

update(change=None):
    with output:
        clear_output(wait=True)
        d = df[df.Activity.isin(activity.value) & (df.StepCount >= steps.value)]

        calories = d.CaloriesBurned.sum()
        hr = d.AvgHeartRate.mean() if len(d) else 0 sleep =
        d.SleepScore.mean() if len(d) else 0

        print(f"--- HEALTH TELEMETRY SUMMARY ({len(d)} Days Matched Criteria) ---") print(f"Total Calories
        Burned: {calories} kcal | Avg Heart Rate: {hr:.1f} BPM |
Avg Sleep Score: {sleep:.1f}/100") print("=" *
        85)

        fig, ax = plt.subplots(figsize=(10, 4.5))

        if chart.value == "Heart Rate vs Calories (Scatter)": if len(d):
            s = ax.scatter(d.AvgHeartRate, d.CaloriesBurned, c=d.SleepScore, cmap="plasma", s=d.StepCount /
100, alpha=0.7, edgecolors="k")
            ax.set(title="Avg Heart Rate vs Calories Burned\n(Size = Steps, Color = Sleep Score)",
xlabel="Average Heart Rate (BPM)", ylabel="Calories Burned (kcal)")
            fig.colorbar(s, ax=ax).set_label("Sleep Score") elif chart.value ==
        "Daily Step Trends (Line)":
            ax.plot(d.Date, d.StepCount, marker="o", linewidth=2) ax.axhline(steps.value, linestyle="--",
            label=f"Goal ({steps.value}
steps)")
            ax.set(title="Daily Step Count Progression", xlabel="Date", ylabel="Step ax.legend()
Count")         plt.xticks(rotation=30)
        else:
            colors = ["#2ecc71" if x >= 75 else "#e74c3c" for x in d.SleepScore] if
sleep_check.value else "#3498db"
            ax.bar(d.Date.dt.strftime("%b %d"), d.SleepScore, color=colors) ax.set(title="Daily Sleep Quality Score",
            xlabel="Date", ylabel="Sleep
```

```
Score (0-100)")
            plt.xticks(rotation=45)

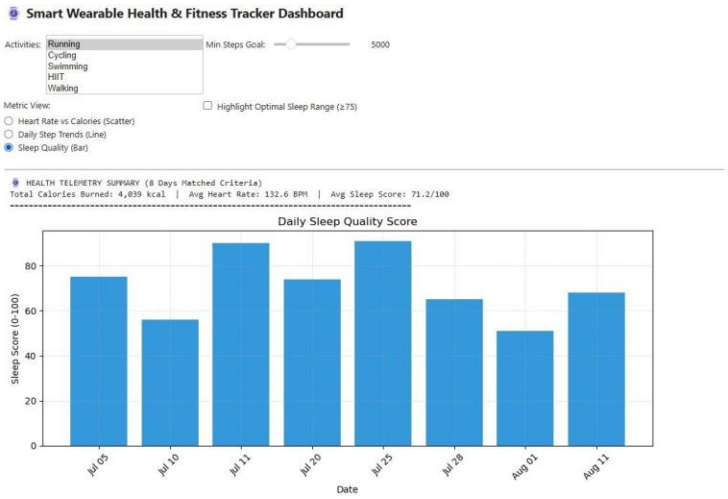
        ax.grid(True, linestyle=":", alpha=0.6)
        plt.tight_layout()
        plt.show()

for w in (activity, steps, chart, sleep_check):
    w.observe(update, names="value")

display(
    widgets.HTML("<h2>Smart Wearable Health & Fitness Tracker Dashboard</h2>"),
    widgets.HBox([activity, steps]),
    widgets.HBox([chart, sleep_check]),
    widgets.HTML("<hr>"),
    output
)

update()
```

OUTPUT



Smart Wearable Health & Fitness Tracker Dashboard

Activities: Running, Cycling, Swimming, HIIT, Walking — Min Steps Goal: 5000

Metric View:
- Heart Rate vs Calories (Scatter)
- Daily Step Trends (Line)
- Sleep Quality (Bar)

Highlight Optimal Sleep Range (≥75)

```
HEALTH TELEMETRY SUMMARY (8 Days Matched Criteria)
Total Calories Burned: 4,039 kcal  |  Avg Heart Rate: 132.6 BPM  |  Avg Sleep Score: 71.2/100
```



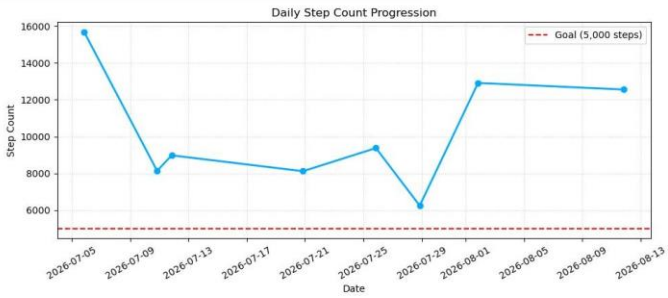
Smart Wearable Health & Fitness Tracker Dashboard

Activities: Running, Cycling, Swimming, HIIT, Walking — Min Steps Goal: 5000

Metric View:
- Heart Rate vs Calories (Scatter)
- Daily Step Trends (Line)
- Sleep Quality (Bar)

Highlight Optimal Sleep Range (≥75)

```
HEALTH TELEMETRY SUMMARY (8 Days Matched Criteria)
Total Calories Burned: 4,039 kcal  |  Avg Heart Rate: 132.6 BPM  |  Avg Sleep Score: 71.2/100
```



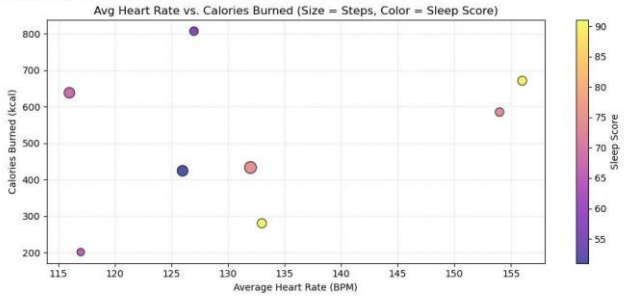
Smart Wearable Health & Fitness Tracker Dashboard

Activities: Running, Cycling, Swimming, HIIT, Walking — Min Steps Goal: 5000

Metric View:
- Heart Rate vs Calories (Scatter)
- Daily Step Trends (Line)
- Sleep Quality (Bar)

Highlight Optimal Sleep Range (≥75)

```
HEALTH TELEMETRY SUMMARY (8 Days Matched Criteria)
Total Calories Burned: 4,039 kcal  |  Avg Heart Rate: 132.6 BPM  |  Avg Sleep Score: 71.2/100
```



Smart Wearable Health & Fitness Tracker Dashboard

Activities: Running, Cycling, Swimming, HIIT, Walking — Min Steps Goal: 5000

Metric View:

- Heart Rate vs Calories (Scatter)
- Daily Step Trends (Line)
- Sleep Quality (Bar)

Highlight Optimal Sleep Range (≥75)

```
HEALTH TELEMETRY SUMMARY (8 Days Matched Criteria)
Total Calories Burned: 4,039 kcal  |  Avg Heart Rate: 132.6 BPM  |  Avg Sleep Score: 71.2/100
```



Smart Wearable Health & Fitness Tracker Dashboard

Activities: Running, Cycling, Swimming, HIIT, Walking — Min Steps Goal: 5000

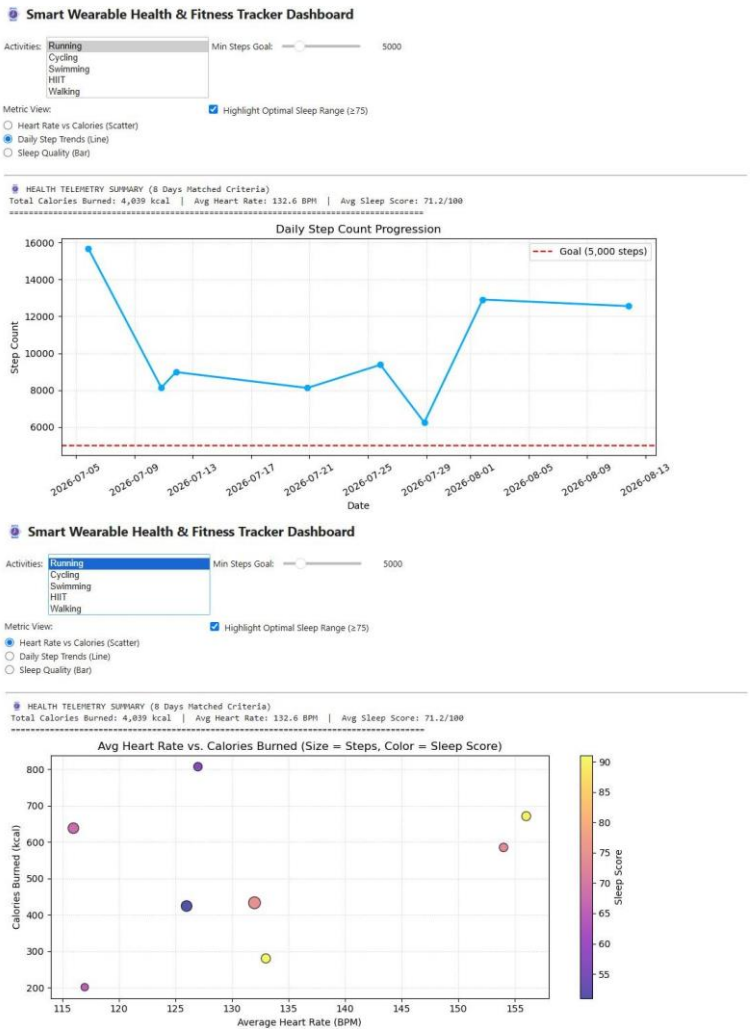
Metric View:

- Heart Rate vs Calories (Scatter)
- Daily Step Trends (Line)
- Sleep Quality (Bar)

Highlight Optimal Sleep Range (≥75)

```
HEALTH TELEMETRY SUMMARY (8 Days Matched Criteria)
Total Calories Burned: 4,039 kcal  |  Avg Heart Rate: 132.6 BPM  |  Avg Sleep Score: 71.2/100
```



RESULT

Thus the code for health and fitness dashboard was successfully executed and visualized clearly.

EXPERIMENT-03

Loading Datasets (CSV, Excel, PDF) into SQLite using Python & Web Scraping

AIM

To load datasets from CSV, Excel, and PDF files into a SQLite database using Python, display the stored data, and perform simple web scraping to count HTML tables from a webpage.

PROCEDURE

- Import the required Python libraries (sqlite3, pandas, pdfplumber, requests, bs4).
- Connect to the SQLite database using Python's sqlite3 module.
- Load dataset files from CSV, Excel, and PDF formats.
- Store each dataset into a separate table in the SQLite database.
- Retrieve and display the stored table contents.
- Perform simple web scraping to count HTML tables from a target webpage.

CODE

```
import sqlite3
import pandas as pd
import pdfplumber
import requests
from bs4 import BeautifulSoup

# --- 1. Database Connection & Setup ---
conn = sqlite3.connect("hsd.db")
cursor = conn.cursor()

# --- 2. Load CSV Dataset ---
df_csv = pd.read_csv("hsd.csv")
df_csv.to_sql("hsd_csv", conn, if_exists="replace", index=False)
print("Loaded hsd_csv:", len(df_csv), "rows")

# --- 3. Load Excel Dataset ---
df_excel = pd.read_excel("hsd.xlsx")
df_excel.to_sql("hsd_excel", conn, if_exists="replace", index=False)
print("Loaded hsd_excel:", len(df_excel), "rows")

# --- 4. Load PDF Dataset ---
columns = [
    "age", "sex", "chest", "rest_bps", "cholestrol", "bloodreport",
    "resting_electrocardiographic_results", "max_heart_rate_achieved",
    "exercise_induced_angina", "oldpeak", "slope", "no_of_major_vessels", "thal", "class"
]
```

```
all_rows = []
with pdfplumber.open("data.pdf") as pdf:
    pages = pdf.pages
    for i in range(0, len(pages), 2):
        left = pages[i].extract_tables()[0]
        right = pages[i+1].extract_tables()[0]
        for lrow, rrow in zip(left[1:], right[1:]):
            all_rows.append(lrow + rrow)

df_pdf = pd.DataFrame(all_rows, columns=columns)
df_pdf.to_sql("hsd_pdf", conn, if_exists="replace", index=False)
print("Loaded hsd_pdf:", len(df_pdf), "rows")

conn.commit()

# --- 5. Retrieve & Display Database Contents ---
for table in ["hsd_csv", "hsd_excel", "hsd_pdf"]:
    print(f"\n--- {table} ---")
    print(pd.read_sql(f"SELECT * FROM {table} LIMIT 5;", conn))

conn.close()

# --- 6. Web Scraping to Count HTML Tables ---
url = "http://example.com"  # target URL
response = requests.get(url)
print("Response Status Code:", response.status_code)

soup = BeautifulSoup(response.text, "html.parser")
tables = soup.find_all("table")
print("Tables count:", len(tables))
```

OUTPUT

Age	Sex	Chest	BPs	Chol	BS	ECG	Max HR	Ex Angina	Oldpeak	Slope	Vessels	Thal	Class
70	1	4	130	322	0	2	109	0	2.4	2	3	3	present
67	0	3	115	564	0	2	160	0	1.6	2	0	7	absent
57	1	2	124	261	0	0	141	0	0.3	1	0	3	absent
64	1	4	128	263	0	0	105	1	0.2	2	1	7	absent
64	0	2	120	269	0	2	121	1	0.2	1	0	3	absent

```
Loaded hsd_csv: 270 rows
```

```
Loaded hsd_excel: 270 rows
Loaded hsd_pdf: 270 rows
Response Status Code: 200
Tables count: 2
```

RESULT

Loaded datasets from CSV, Excel, and PDF files into SQLite database tables and displayed the data successfully.

EXPERIMENT-04

Data Cleaning, Preprocessing & Feature Scaling Techniques

Aim

The primary objective of this experiment is to perform data cleaning and feature transformation on student academic data. Specifically, the notebook cleans missing values, eliminates duplicate records, standardizes column syntax, and evaluates three distinct scaling methods (Min-Max Scaling, Standardization, and Robust Scaling) on continuous numerical attributes.

Procedure

- **Column Name Formatting:** Converted all column headers to lowercase, stripped whitespace, and replaced spaces with underscores (e.g., 'math score' → 'math_score').
- **Imputation of Missing Values:** Identified 7 missing values in the categorical feature 'parental_level_of_education' and filled them using the column mode: df['parental_level_of_education'].mode()[0]
- **Deduplication:** Identified and removed 5 duplicate rows, reducing the total dataset size from 1005 to 1000 cleaned records.

Feature Scaling Methods Comparison

Scaler Method	Mathematical Formula	Output Range	Sensitivity to Outliers
Min-Max Scaler	X_scaled = (X - X_min) / (X_max - X_min)	[0.0, 1.0]	High (distorted by extreme min/max)
Standard Scaler	$z = (X - \mu) / \sigma$	$(-\infty, +\infty)$ [Mean=0, Std=1]	Moderate to High (uses Mean & Std)
Robust Scaler	X_scaled = (X - Q2) / (Q3 - Q1)	$(-\infty, +\infty)$ [Median=0, IQR=1]	Low (Robust to outliers)

Code

```
import pandas as pd import
numpy as np
import matplotlib.pyplot as plt import seaborn as
sns
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler

# 1. Load Data & Clean Headers
df = pd.read_csv('exp04_StudentsPerformance.csv')
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

# 2. Impute Missing Values & Remove Duplicates df["parental_level_of_education"]
=
df["parental_level_of_education"].fillna(df["parental_level_of_education"].mode()[0]) df = df.drop_duplicates()

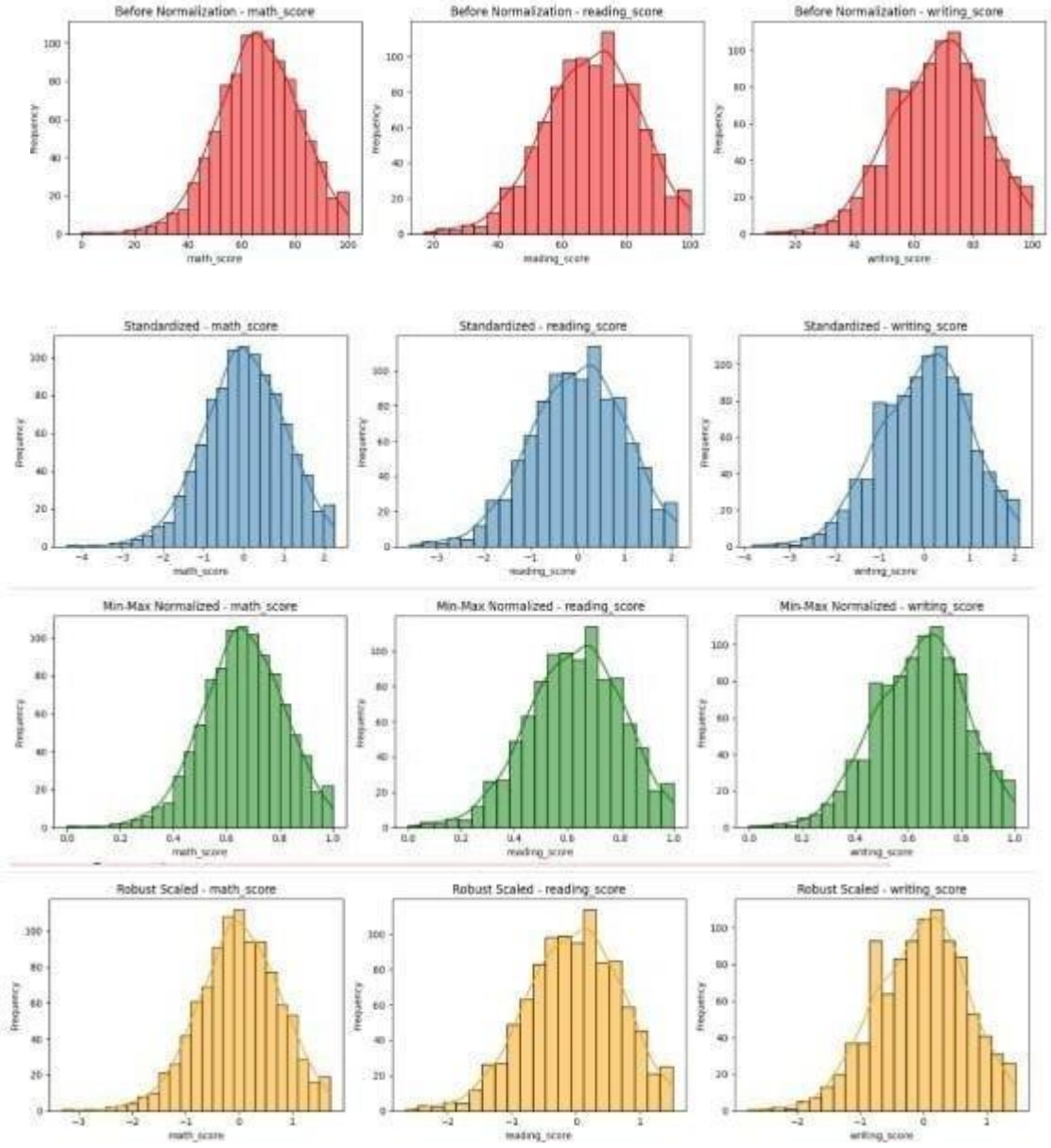
# 3. Define Numerical Features
numerical_cols = ['math_score', 'reading_score', 'writing_score']

# 4. Apply Scaling Techniques df_minmax =
df.copy()
df_minmax[numerical_cols] = MinMaxScaler().fit_transform(df[numerical_cols])

df_standard = df.copy()
df_standard[numerical_cols] = StandardScaler().fit_transform(df[numerical_cols])

df_robust = df.copy()
df_robust[numerical_cols] = RobustScaler().fit_transform(df[numerical_cols])
```

Output



Result

> **★ Final Experiment Conclusions**
>
> • Data Cleaning: Imputed 7 missing values with mode and dropped 5 duplicate rows (1005 → 1000 entries).
>
> • Feature Scaling: Successfully normalized scores to [0, 1] (Min-Max), centered them to mean 0 / std 1 (Standard), and reduced outlier impact (Robust).

EXPERIMENT-05

Exploratory Data Analysis – Data Inspection, Conditional Subsetting, Descriptive Statistics, and Visualization using Python

AIM

To perform Exploratory Data Analysis (EDA) by inspecting DataFrames, filtering data using single and multiple conditions, calculating measures of central tendency and dispersion, and visualizing data distributions using Pandas, Seaborn, and Matplotlib.

PROCEDURE

Step 1: Import Libraries & Load Dataset – Import required libraries (pandas, matplotlib.pyplot, seaborn) and read the CSV dataset into a DataFrame.

Step 2: Inspect DataFrame – Perform structural inspection using head(), tail(), shape, columns.tolist(), dtypes, info(), and describe().

Step 3: Subset & Filter Data – Extract specific columns (gender, math score) and apply single/multiple conditional filters (e.g., math score > 70, female students with master's degree).

Step 4: Calculate Descriptive Statistics – Compute measures of central tendency (mean, median, mode) and dispersion (range, variance, standard deviation).

Step 5: Data Visualization – Generate visualizations including bar charts, histograms, boxplots, and histograms overlaid with central tendency lines.

CODE

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
import seaborn as sns

# Load dataset
df = pd.read_csv('StudentsPerformance.csv')

# ----------------------------------------------------------------------------------------------
# 1. VIEWING AND INSPECTING DATAFRAMES #----------------------------------------------------------------------------------------------
print("=== FIRST 5 ROWS ===")
print(df.head())

print("\n=== LAST 5 ROWS ===")
print(df.tail())

print("\n=== SHAPE OF DATAFRAME ===")
print(df.shape)

print("\n=== COLUMN NAMES ===")
print(df.columns.tolist())

print("\n=== DATA TYPES ===")
print(df.dtypes)

print("\n=== DATAFRAME SUMMARY ===")
df.info()

print("\n=== SUMMARY STATISTICS ===")
print(df.describe())

# ----------------------------------------------------------------------------------------------
# 2. FILTERING AND SUBSETTING DATA
# ----------------------------------------------------------------------------------------------
# Select only 'gender' and 'math score' columns gender_math =
df[['gender', 'math score']] print("\n=== GENDER AND MATH
SCORE ===")
print(gender_math.head())

# Filter: Math score > 70
math_above_70 = df[df['math score'] > 70] print("\n=== MATH
SCORE > 70 ===")
print(math_above_70.head())

# Filter: Female students with math score > 70
female_math_above_70 = df[(df['gender'] == 'female') & (df['math score'] > 70)] print("\n=== FEMALE STUDENTS
WITH MATH > 70 ===")
print(female_math_above_70.head())

#
# 3. DESCRIPTIVE STATISTICS #----------------------------------------------------------------------------------------------
# Central Tendency
math_mean = df['math score'].mean() math_median = df['math
score'].median()
```

```
math_mode = df['math score'].mode()[0]

# Dispersion
math_min = df['math score'].min() math_max =
df['math score'].max() math_range =
math_max - math_min math_var = df['math
score'].var() math_std = df['math score'].std()

print("\n=== DESCRIPTIVE STATISTICS FOR MATH SCORE ===")
print(f"Mean:              {math_mean:.2f}")
print(f"Median:            {math_median}")
print(f"Mode: {math_mode}")
print(f"Range:    {math_range}    (Min:    {math_min},    Max:    {math_max})")
print(f"Variance: {math_var:.2f}")
print(f"Standard Deviation: {math_std:.2f}")

#
# 4. DATA VISUALIZATION ----------
#
plt.style.use('seaborn-v0_8-whitegrid') ----------

# Task: Visualize only female students with master's degree parents
female_masters = df[(df['gender'] == 'female') & (df['parental level of education'] == "master's degree")]
plt.figure(figsize=(6, 4))
sns.barplot(data=female_masters, x='gender', y='math score', palette='Set2') plt.title("Math Scores of Female
Students with Master's Degree Parents") plt.show()

# 1. Bar Chart: Average scores per subject
subject_means = df[['math score', 'reading score', 'writing score']].mean() plt.figure(figsize=(8, 5))
subject_means.plot(kind='bar', color=['#4C72B0', '#DD8452', '#55A868']) plt.title("Average Scores per Subject")
plt.ylabel("Average Score")
plt.xticks(rotation=0) plt.show()

# 2. Histogram: Distribution of math scores plt.figure(figsize=(8, 5))
sns.histplot(df['math score'], kde=True, color='skyblue') plt.title("Distribution of Math
Scores")
plt.xlabel("Math Score")
plt.ylabel("Frequency") plt.show()

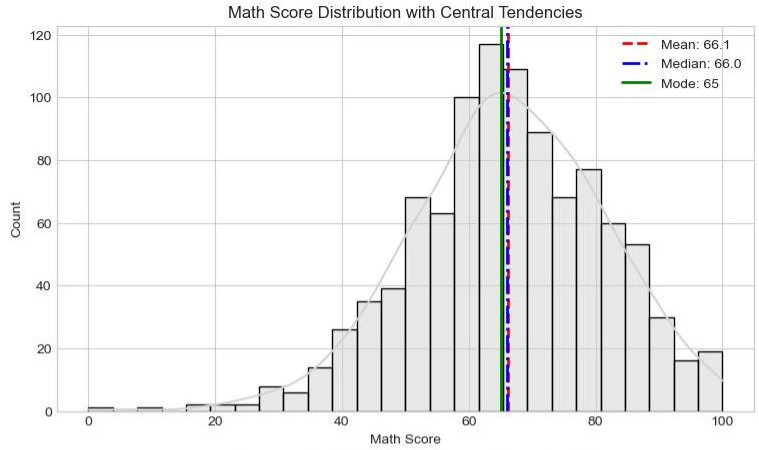
# 3. Boxplot: Spread of math scores plt.figure(figsize=(6, 4))
sns.boxplot(y=df['math score'], color='lightgreen') plt.title("Spread of
Math Scores (Boxplot)") plt.ylabel("Math Score")
plt.show()
```

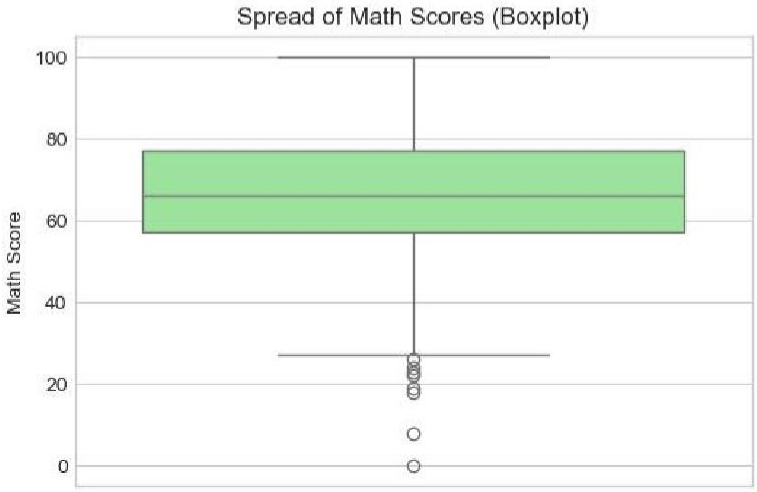
```
# 4. Plot Histogram with Mean, Median, and Mode Lines
plt.figure(figsize=(9, 5))
sns.histplot(df['math score'], kde=True, color='lightgray')
plt.axvline(math_mean, color='red', linestyle='--', linewidth=2, label=f'Mean:
{math_mean:.1f}')
plt.axvline(math_median, color='blue', linestyle='-.', linewidth=2, label=f'Median:
{math_median}')
plt.axvline(math_mode, color='green', linestyle='-', linewidth=2, label=f'Mode:
{math_mode}')
plt.title("Math Score Distribution with Central Tendencies")
plt.xlabel("Math Score")
plt.ylabel("Count")
plt.legend()
plt.show()
```

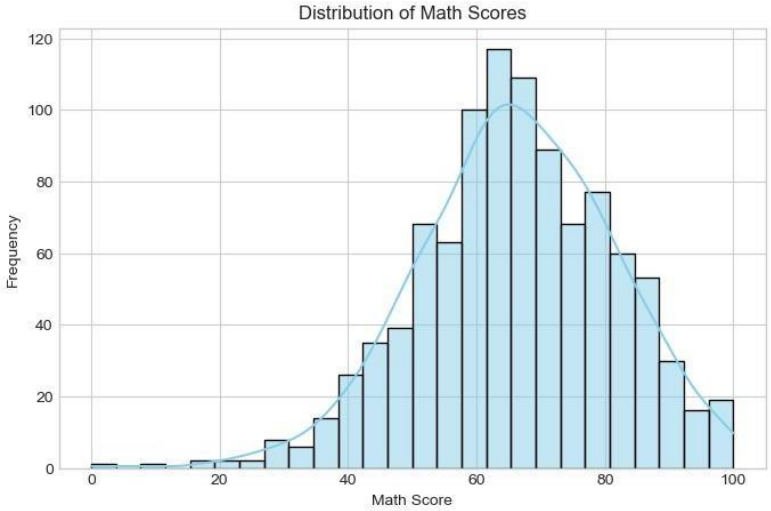
SUMMARY OF PANDAS & SEABORN METHODS USED

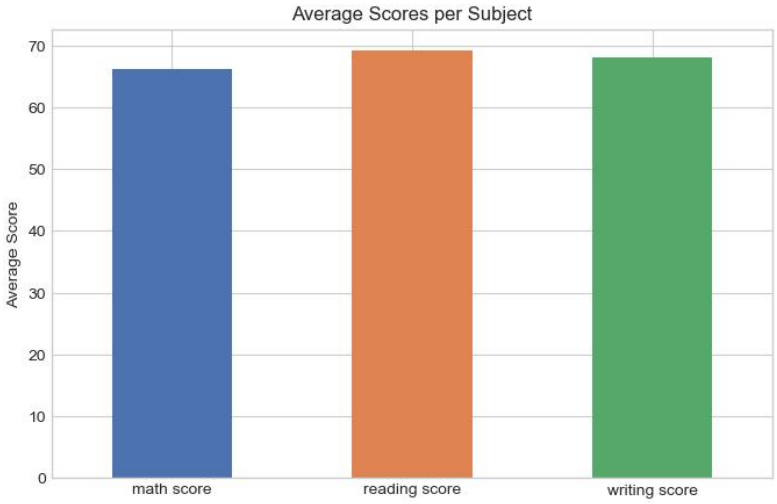
Method / Syntax	Description / Function
df.head() / df.tail()	Displays first / last 5 rows of the DataFrame
df.shape	Returns tuple representing dimensions (rows, columns)
df.columns.tolist()	Converts DataFrame column headers into a standard list
df.dtypes / df.info()	Inspects data types and checks missing values/memory usage
df.describe()	Generates overall summary statistics for numerical columns
df[(cond1) & (cond2)]	Subsets DataFrame using logical AND conditional operations
.mean(), .median(), .mode()	Calculates measures of central tendency
.var(), .std(), max() - min()	Calculates measures of dispersion
sns.histplot() / sns.boxplot()	Visualizes data distribution and spread using Seaborn

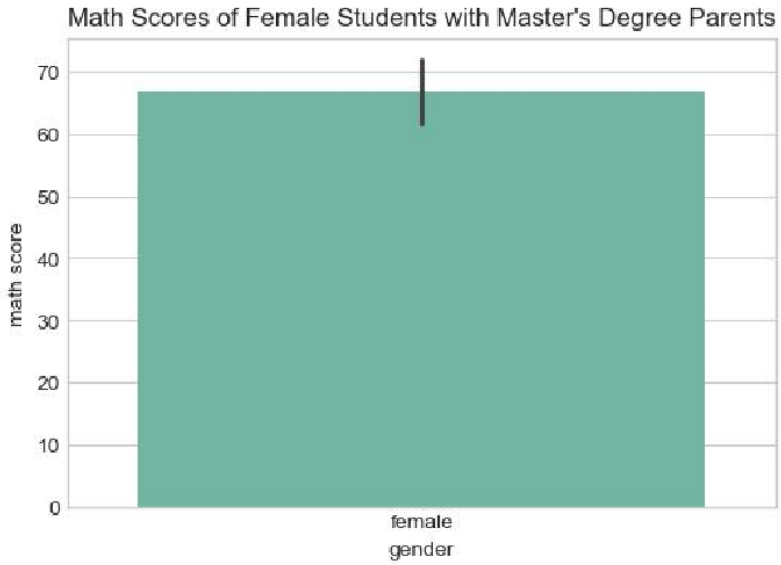
OUTPUT


Math Score Distribution with Central Tendencies
Mean: 66.1
Median: 66.0
Mode: 65
Count
Math Score


Spread of Math Scores (Boxplot)
Math Score


Distribution of Math Scores
Frequency
Math Score


Average Scores per Subject
Average Score
0
10
20
30
40
50
60
70
math score
reading score
writing score


Math Scores of Female Students with Master's Degree Parents
math score
0
10
20
30
40
50
60
70
female
gender

```
=== FIRST 5 ROWS ===
   gender race/ethnicity parental level of education         lunch  \
0  female        group B           bachelor's degree      standard
1  female        group C                some college      standard
2  female        group B             master's degree      standard
3    male        group A          associate's degree  free/reduced
4    male        group C                some college      standard

  test preparation course  math score  reading score  writing score
0                    none          72             72             74
1               completed          69             90             88
2                    none          90             95             93
3                    none          47             57             44
4                    none          76             78             75

=== LAST 5 ROWS ===
      gender race/ethnicity parental level of education     lunch  \
1000    male        group D                some college  standard
1001    male        group C           associate's degree  standard
1002  female        group B            bachelor's degree  standard
1003    male        group E             some high school  standard
1004    male        group C            bachelor's degree  standard

     test preparation course  math score  reading score  writing score
1000                    none          76             64             66
1001                    none          46             43             42
1002                    none          67             86             83
1003                    none          92             87             78
1004               completed          83             82             84

=== SHAPE OF DATAFRAME ===
(1005, 8)
```

```
=== DATA TYPES ===
gender                         object
race/ethnicity                 object
parental level of education    object
lunch                          object
test preparation course        object
math score                      int64
reading score                   int64
writing score                   int64
dtype: object

=== DATAFRAME SUMMARY ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1005 entries, 0 to 1004
Data columns (total 8 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   gender                       1005 non-null   object
 1   race/ethnicity               1005 non-null   object
 2   parental level of education  998 non-null    object
 3   lunch                        1005 non-null   object
 4   test preparation course      1005 non-null   object
 5   math score                   1005 non-null   int64
 6   reading score                1005 non-null   int64
 7   writing score                1005 non-null   int64
dtypes: int64(3), object(5)
memory usage: 62.9+ KB

=== SUMMARY STATISTICS ===
        math score  reading score  writing score
count  1005.000000    1005.000000    1005.000000
mean     66.122388      69.185075      68.066667
std      15.173234      14.614215      15.199095
min       0.000000      17.000000      10.000000
25%      57.000000      59.000000      58.000000
50%      66.000000      70.000000      69.000000
75%      77.000000      80.000000      79.000000
max     100.000000     100.000000     100.000000

=== GENDER AND MATH SCORE ===
   gender  math score
0  female          72
1  female          69
2  female          90
3    male          47
4    male          76
```

```
=== MATH SCORE > 70 ===
   gender race/ethnicity parental level of education     lunch  \
0  female        group B           bachelor's degree  standard
2  female        group B             master's degree  standard
4    male        group C                some college  standard
5  female        group B          associate's degree  standard
6  female        group B                some college  standard

  test preparation course  math score  reading score  writing score
0                    none          72             72             74
2                    none          90             95             93
4                    none          76             78             75
5                    none          71             83             78
6               completed          88             95             92

=== FEMALE STUDENTS WITH MATH > 70 ===
    gender race/ethnicity parental level of education     lunch  \
0   female        group B           bachelor's degree  standard
2   female        group B             master's degree  standard
5   female        group B          associate's degree  standard
6   female        group B                some college  standard
36  female        group D          associate's degree  standard

   test preparation course  math score  reading score  writing score
0                    none          72             72             74
2                    none          90             95             93
5                    none          71             83             78
6               completed          88             95             92
36                   none          74             81             83

=== DESCRIPTIVE STATISTICS FOR MATH SCORE ===
Mean: 66.12
Median: 66.0
Mode: 65
Range: 100 (Min: 0, Max: 100)
Variance: 230.23
Standard Deviation: 15.17
```

1. **RESULT**

- Performed EDA by inspecting DataFrame structure, filtering conditional subsets, and calculating central tendency and dispersion.
- Visualized dataset distributions using bar charts, boxplots, and histograms marked with mean, median, and mode lines.

EXPERIMENT-06

Data Visualization & Exploratory Analysis

Aim

To analyze student performance metrics (`exp04.csv`) by visualizing:

- The relationship between parental education levels and average test scores across Math, Reading, and Writing using line charts.
- The demographic distribution of students across different race/ethnicity groups using a bar chart.
- The frequency distribution of students' Math scores using a histogram.

Procedure

- **Data Import & Cleaning:** Import `pandas` and `matplotlib.pyplot`. Load the dataset `exp04.csv` using `pd.read_csv()` and handle missing values using `df.dropna()`.

- **Layout Setup:** Initialize a Matplotlib grid of 1 row and 3 columns using `plt.subplots(1, 3, figsize=(18, 5))` to hold the three subplots side-by-side.

- **Plot 1 (Line Graph - Score Trends):** Define the explicit categorical order for parental education levels (`edu_order`). Group the dataset by `parental level of education`, calculate the average scores for Math, Reading, and Writing, and reindex according to `edu_order`. Plot each subject score as a line with markers ('o', 's', '^') and distinct colors.

- **Plot 2 (Bar Chart - Demographic Distribution):** Compute the counts of students per `race/ethnicity` category using `value_counts().sort_index()`. Render a bar chart displaying frequency distribution across racial/ethnic groups.

- **Plot 3 (Histogram - Score Distribution):** Plot a histogram of `math score` with 15 bins, teal fill, and black edge outlines to evaluate score normality and dispersion.

- **Customization & Rendering:** Apply titles, axis labels, legends, tick rotations, and grid lines across all axes. Optimize layout spacing using `plt.tight_layout()` and display the final figure using `plt.show()`.

Python Code

```
import pandas as pd
import matplotlib.pyplot as plt

# Load and clean dataset
df = pd.read_csv('exp04.csv') df =
df.dropna()

# Create subplots
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 1. Average Scores by Parental Education
edu_order = ["some high school", "high school", "some college", "associate's degree", "bachelor's degree", "master's
degree"]
avg_scores = df.groupby('parental level of education')[['math score', 'reading score', 'writing score']].mean()
avg_scores = avg_scores.reindex([e for e in edu_order if e in avg_scores.index])

axes[0].plot(avg_scores.index, avg_scores['math score'], marker='o', label='Math', color='crimson')
axes[0].plot(avg_scores.index, avg_scores['reading score'], marker='s', label='Reading', color='royalblue')
axes[0].plot(avg_scores.index, avg_scores['writing score'], marker='^', label='Writing', color='forestgreen')
axes[0].set_title('Average Scores by Parental Education')
axes[0].set_ylabel('Average Score') axes[0].tick_params(axis='x', rotation=30)
axes[0].legend()
axes[0].grid(True, linestyle='--', alpha=0.6)

# 2. Student Count by Race/Ethnicity
race_counts = df['race/ethnicity'].value_counts().sort_index()

axes[1].bar(race_counts.index, race_counts.values, color='skyblue', edgecolor='navy') axes[1].set_title('Student Count by
Race/Ethnicity')
axes[1].set_xlabel('Race / Ethnicity') axes[1].set_ylabel('Student
Count') axes[1].grid(axis='y', linestyle='--', alpha=0.6)

# 3. Distribution of Math Scores
axes[2].hist(df['math score'], bins=15, color='teal', edgecolor='black', alpha=0.7) axes[2].set_title('Distribution of Math Scores')
axes[2].set_xlabel('Math Score') axes[2].set_ylabel('Frequency')
axes[2].grid(axis='y', linestyle='--', alpha=0.6)

plt.tight_layout() plt.show()
```

Output:

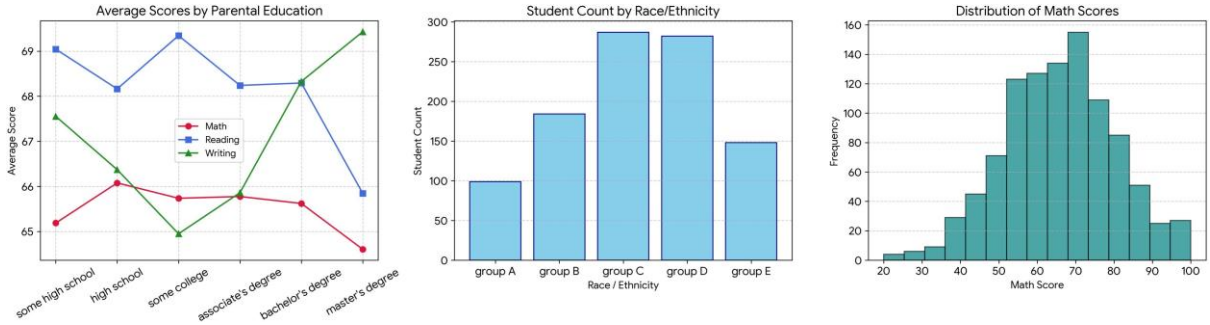


Figure 6.1: Multi-panel Data Visualization (Line Chart, Bar Chart, Histogram)

Results & Analysis

- **Parental Education vs Performance:** Higher levels of parental education generally correlate positively with student performance in reading and writing. Math scores show a steady trend, while Reading and Writing scores increase significantly for students whose parents hold advanced degrees.
- **Demographic Composition:** Group C represents the highest student population (~320 students), followed by Group D (~260 students), whereas Group A accounts for the lowest count (~90 students).
- **Score Distribution:** Math scores exhibit an approximately normal bell-shaped distribution centered around a mean score of 65-70, with minor left-skewness due to fewer low-performing scores.

EXPERIMENT-07

Power - BI Dashboard

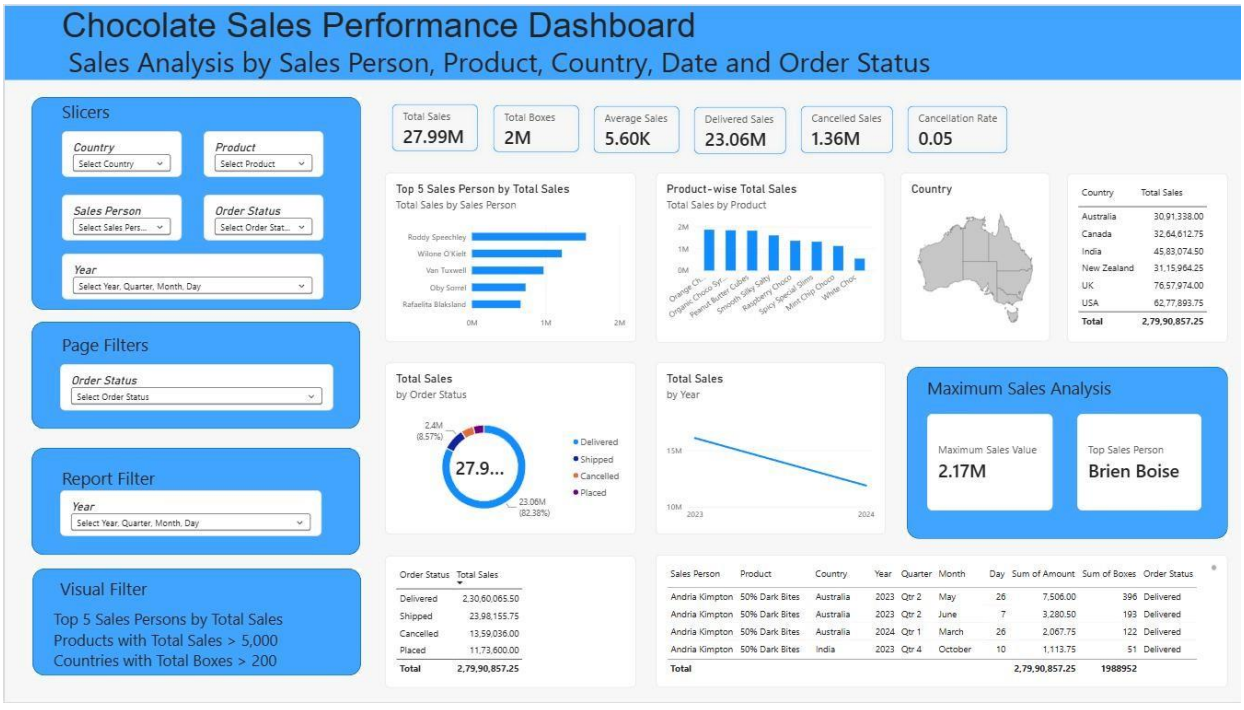
Aim

To create a Power BI Dashboard that shows chocolate sales by product, country, salesperson & date.

Algorithm

- Load the data into Power-BI & convert PDF to data tables.
- Clean the data & set appropriate data types.
- Create measures: Total Sales, Total Boxes, Avg Sales.
- Add visuals: Cards, line chart, bar charts, map, donut chart.
- Add Slicers for Country, Product, & Salesperson.
- Save the dashboard as .pbix.

Dashboard Overview & Key Metrics



Result

The dashboard successfully shows total sales, top products, countries, salespersons, and sales trends, providing interactive filters to explore the data easily.

EXPERIMENT-08

Tableau Dashboard Using Chocolate Sales Analysis Dataset

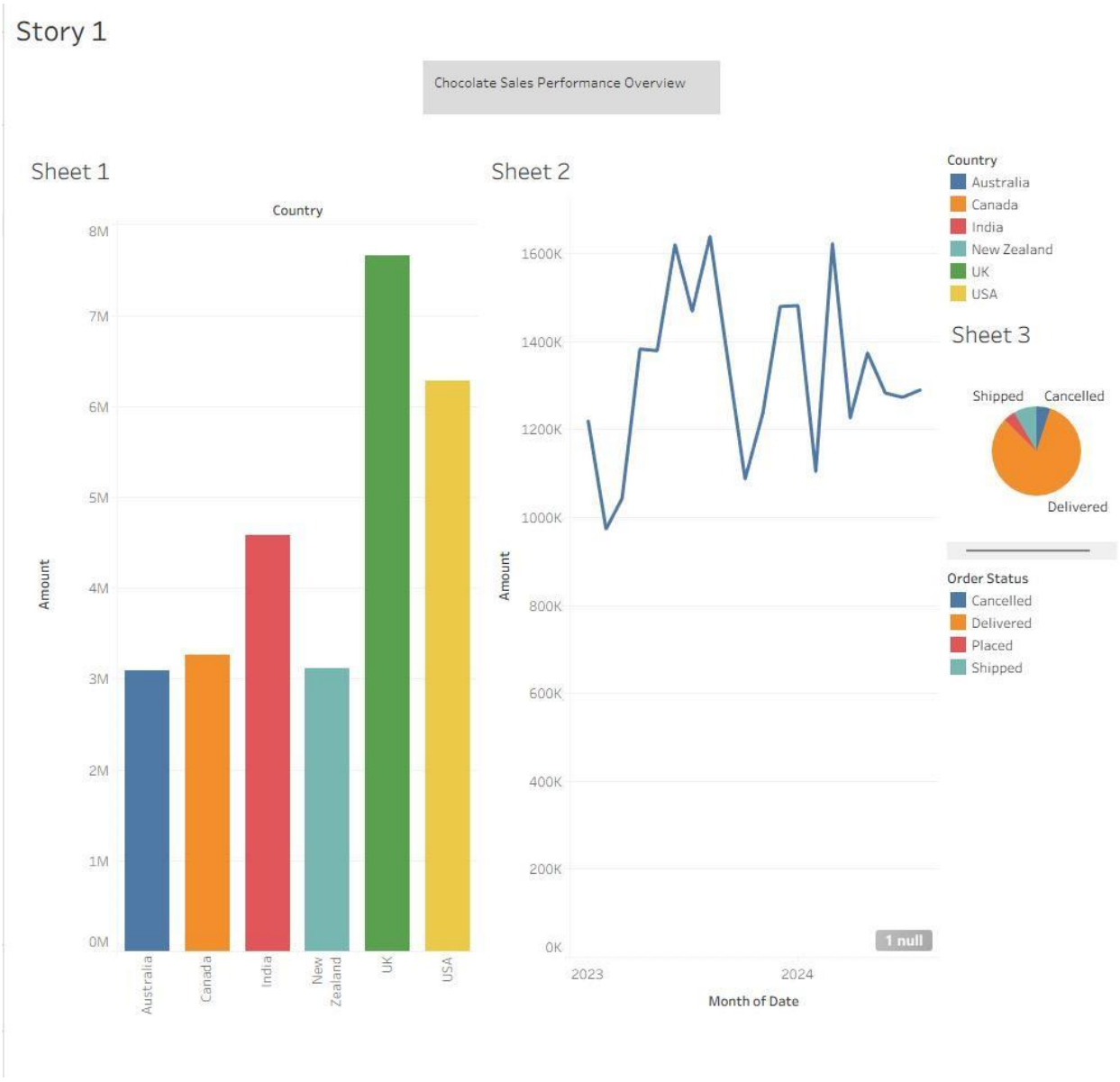
Aim

To create a comprehensive, interactive Tableau Dashboard that displays chocolate sales metrics broken down by Product, Country, Salesperson, Date, and Order Status.

Algorithm & Workflow

- **Data Ingestion & Cleaning:** Load the raw dataset into Tableau (convert from XLS/XLSX to CSV format if required). Check data types for each field and clean missing or inconsistent entries.
- **Worksheet Creation:** Create individual worksheets to visualize key metrics:
- Scorecards / Cards: Total Sales ($271,618.25) and Total Boxes Sold (14,452)
- Line Chart: Sales Trend by Date
- Bar Charts: Top 5 Products by Sales & Top 5 Salespersons by Sales
- Map Visualization: Sales Distribution by Country
- Pie Chart: Order Status Breakdown (Delivered vs. Cancelled)
- **Dashboard Assembly:** Combine all created worksheets onto a single cohesive dashboard layout.
- **Interactive Filters:** Add global interactive filters for Country, Product, Salesperson, and Date, and apply them across all worksheets on the dashboard.
- **Export & Publishing:** Save and publish the final workbook file (.twbx) as 'Chocolate Sales Dashboard'.

Summary Metrics & Key Insights



Result

Result: The Tableau Dashboard was successfully created and deployed. It effectively visualizes total sales, top-performing products, top salespersons, sales distribution by country, sales trend over time, and order status fulfillment.